Variable-size Neural Networks for 3D Timepix Hit Grouping and Unsupervised Particle Clustering

Concrete architecture, training design, autoencoder options, and SOTA survey

Technical note for raw `.t3pa` Timepix/TPX3 hit streams

May 1, 2026

Abstract

The first task is to reproduce and improve DBSCAN-style particle grouping directly from raw time-quantized Timepix/TPX3 hits. The input is a variable-size set of hits sorted by time; the output is a variable number of particle instances, each of variable size, ranging from a single hit to hundreds of hits. This report explains in detail how such a neural network physically looks: a shared per-hit encoder, local 3D graph/message-passing layers, and per-hit outputs that produce either an embedding, links, object-condensation variables, or slot/query masks. The second task is unlabeled grouping of already separated particles according to 3D trajectory shape, time profile, and energy/ToT behavior. For that task, this report surveys point-cloud encoders, autoencoders, variational and deep-embedded clustering, masked autoencoders, and contrastive representation learning. The recommended practical design is: **Pass 1: EdgeTrackNet-Tiny or CondensationTrackNet for hit-to-particle grouping; Pass 2: ParticleAE/ParticleEmbed with HDBSCAN or VaDE/DEC-style deep clustering for unlabeled particle families**. The design keeps variable-size data native, avoids dense images, and is compatible with CPU or small-GPU inference when the number of hits per window is capped or locally pooled.

One-line recommendation

Use a neural network that predicts *relationships or embeddings per hit*, not a fixed-size list of particles. Cluster size is not an output dimension of the neural network; it emerges after connected components, learned condensation, or slot masks. For the second pass, encode each variable-size particle as one latent vector and cluster those latent vectors with density/prototype methods.

---

1 Data abstraction and why variable size is not a problem

A raw hit after parsing the `.t3pa` file can be represented as

$$h_i = (x_i, y_i, t_i, e_i, q_i),$$

where $x_i = \texttt{MatrixIndex} \bmod 256$, $y_i = \lfloor \texttt{MatrixIndex}/256 \rfloor$, t_i is a time coordinate derived from ToA/FToA inside the acquisition or rolling window, $e_i = \log(1 + \mathrm{ToT}_i)$ is the energy-like feature, and q_i can include FToA, acquisition index, duplicate-pixel count, or other local metadata. Timepix3-style detectors natively provide sparse hit readout with simultaneous timing and ToT information, which makes point-cloud processing more natural than dense image processing [Poikela et al., 2014]. Direction reconstruction with Timepix3 has also been studied explicitly as a 3D trajectory problem, not only as a 2D image problem [Mánek et al., 2021].

The input window is a set

$$H = \{h_1, \ldots, h_N\}, \qquad N \text{ is variable.}$$

The output of the first pass is a partition

$$\pi(i) \in \{0, 1, 2, \ldots, K\},$$

where $\pi(i) = 0$ means noise or unassigned hit, and $1 \dots K$ are particle instance IDs. Both N and K are variable. This is exactly why the model should not be a dense CNN with a fixed-size output vector. It should be a *set/graph model* whose weights are reused for every hit.

1.1 How variable-size batching works in practice

There are three standard implementation options. All are simple and mature in PyTorch/TensorFlow-style toolchains:

1. **Ragged set / graph batching.** Concatenate hits from multiple windows into one tensor of shape $[\sum_b N_b, F]$ and keep a `batch_id` vector. Graph libraries such as PyTorch Geometric use exactly this style.
2. **Padding + mask.** Pad every window to $N_{\max}$ hits, keep a binary mask, and ignore padded entries in loss and attention. This is convenient for transformers and ONNX deployment.
3. **Streaming micro-batches.** For live operation, keep a ring buffer for the last 200 ms–250 ms, optionally with overlap. If N is too large, locally pool or cap tokens before running the model.

The network does not need to know in advance whether a particle has 1 hit or 100 hits. It computes features for every hit, then a graph/embedding step decides which hits belong together.

2 Pass 1: neural reproduction and improvement of DBSCAN

2.1 What DBSCAN is doing in 3D

The previous image DBSCAN can be upgraded to 3D by clustering in scaled coordinates,

$$u_i = \left(\frac{x_i}{s_x}, \frac{y_i}{s_y}, \frac{t_i}{s_t}, \gamma e_i \right).$$

DBSCAN finds dense connected components in this space [Ester et al., 1996]. HDBSCAN removes the need for one global density threshold and is useful when track density varies strongly [McInnes et al., 2017]. ST-DBSCAN is explicitly formulated for spatial-temporal data [Birant and Kut, 2007]. These algorithms are the correct baseline and pseudo-label source, but they are not the best final model because their metric is hand-designed and cannot learn detector-specific geometry, time-walk effects, or shape priors.

The neural version should learn a better metric and better link decisions while preserving the variable-size nature of the problem.

2.2 Architecture A: EdgeTrackNet-Tiny

The most concrete first model is an **edge-link network**. It predicts whether two nearby hits belong to the same particle. Final particles are connected components of the predicted same-particle graph.

Figure 1: EdgeTrackNet-Tiny: variable-size input, local link prediction, variable-size output through connected components.

Input tensor. For each hit use 6–10 features:

$$f_i = [x/255,\ y/255,\ t/T,\ \log(1 + \mathrm{ToT})/c,\ \mathrm{FToA}/30,\ \Delta t_{\mathrm{prev}},\ \Delta t_{\mathrm{next}}].$$

The hits may be sorted by time. Sorting is useful for computing Δt, but the model itself should be permutation-equivariant: shuffling hits should shuffle outputs in the same way.

Local graph. Build a local graph with $k = 8$–32 nearest neighbors in scaled (x, y, t) space. This is the only non-neural geometric step. It is fast because it is local and the detector grid is small. The edge feature for a pair (i, j) is

$$g_{ij} = [f_i, f_j, f_i - f_j, \|r_i - r_j\|, \Delta t_{ij}, |e_i - e_j|].$$

Network layers. A concrete CPU-friendly version is:

$$\begin{aligned} \text{HitMLP}: &\quad F \to 64 \to 64, \\ \text{EdgeMLP}: &\quad 2 \cdot 64 + 6 \to 64 \to 32 \to 1, \\ \text{OptionalMessage}: &\quad \text{one EdgeConv/DGCNN-style update}, \end{aligned}$$

where EdgeConv-style local message passing is a strong point-cloud baseline [Wang et al., 2019]. The output $p_{ij} \in [0, 1]$ estimates whether two hits belong to the same particle. Edges with $p_{ij} > \tau$ become a graph; connected components are the particle candidates.

Why this handles 1-hit particles. A single-hit particle has no high-confidence links. It remains as an isolated connected component if its hit-objectness score is high. Noise is suppressed by an additional per-hit noise/objectness head.

Table 1: Recommended first implementation of EdgeTrackNet-Tiny.

Component	Concrete choice
Input window	200 ms–250 ms, optional 50% overlap
Max hits per pass	2048–8192; above that, local pooling or top-k + reservoir sampling
Hit features	$x, y, t, \log(1 + \text{ToT}), \text{FToA}, \Delta t$
Graph	kNN or radius graph in scaled (x, y, t); $k = 8$–32
Backbone	shared MLP + 1–2 EdgeConv/message-passing blocks
Heads	edge same-particle probability, hit noise/objectness score
Readout	thresholded edges + connected components + small cleanup
Parameters	about 0.1–1M
Training target	DBSCAN/HDBSCAN pseudo-labels first; synthetic truth later

2.3 Architecture B: TrackEmbed-Tiny

The second practical model learns an embedding $z_i \in \mathbb{R}^d$ for every hit. Hits from the same particle are close; hits from different particles are far. After the network, a very small DBSCAN/HDBSCAN in embedding space forms instances. This is closest to "neural DBSCAN".

$$h_i \mapsto z_i \in \mathbb{R}^{8\text{–}16}, \qquad d(z_i, z_j) \text{ small iff } i, j \text{ are same particle.}$$

A concrete model:

1. encode each hit with a shared MLP;
2. add local context by one or two kNN message-passing blocks;
3. output embedding z_i, seed score β_i, noise probability n_i, and optional local direction v_i;
4. group by learned embedding distance, optionally biased toward high-seed hits.

This architecture matches object-condensation ideas: each object is represented by high-confidence condensation points and a learned clustering space [Kieseler, 2020]. Object condensation has been used for multi-particle reconstruction in highly granular detectors [Qasim et al., 2021] and recent calorimeter reconstruction work [Matousek et al., 2025].

Loss. With pseudo-label or synthetic instance labels y_i, use a pull-push metric loss:

$$\mathcal{L}_{\text{pull}} = \sum_k \frac{1}{|C_k|} \sum_{i \in C_k} \|z_i - \mu_k\|^2,$$

$$\mathcal{L}_{\text{push}} = \sum_{k \neq l} \max(0, m - \|\mu_k - \mu_l\|)^2,$$

plus noise/objectness loss and optional direction loss. Object-condensation variants replace this with condensation potentials and a learnable β confidence.

2.4 Architecture C: Query/slot particle segmenter

A more transformer-like design uses $K_{\max}$ learned query tokens or slots. Each slot competes for hits and produces one soft mask:

$$M_{ki} = P(\text{hit } i \text{ belongs to slot } k).$$

This is analogous to Slot Attention and query-based object-centric learning [Locatello et al., 2020], and related to transformer/query reconstruction work in tracking and particle-flow settings [Caron et al., 2024]. It naturally returns variable-size particles because each slot mask may contain 1 hit, 100 hits, or no hits.

This is powerful, but I would not start here. It requires setting $K_{\max}$, handling empty slots, and training a set-matching loss. It is excellent as a second-generation model after EdgeTrackNet or TrackEmbed-Tiny is working.

Table 2: Which first-pass architecture to use first.

Model	Best use	Main risk
3D DBSCAN/HDBSCAN	baseline, pseudo-labels, sanity check	manual metric and density scale
EdgeTrackNet-Tiny	fastest neural replacement for DBSCAN; excellent for variable cluster sizes	needs good local graph; long tracks require enough edges
TrackEmbed-Tiny	closest to neural metric clustering; good general first NN	still needs post-clustering threshold/radius
Object Condensation	strongest if synthetic or MC instance truth exists	more complex loss and readout
Query/Slot Segmenter	elegant variable-object transformer model	requires $K_{\max}$ and set matching; slower

3 Training the first pass when DBSCAN is the teacher

3.1 Pseudo-label construction

The current reliable source of grouping information is DBSCAN/HDBSCAN. Use it as a teacher, but only where it is likely correct.

1. Run 3D DBSCAN/HDBSCAN on (x, y, t, e) for many scale settings.
2. Keep clusters stable across nearby parameter settings.
3. Reject ambiguous areas, high-overlap regions, and clusters with inconsistent ToA/ToT profiles.
4. Treat stable clusters as pseudo-ground-truth particles.

5. Treat unstable clusters as unlabeled, not as negative labels.

   This avoids teaching the network every mistake DBSCAN makes.

3.2 Training samples and labels

A training example is one window with N hits and labels:

$$X \in \mathbb{R}^{N\times F}, \quad y \in \{0,1,\dots,K\}^N, \quad N, K \text{ variable.}$$

For EdgeTrackNet, convert labels to edge labels on the local graph:

$$y_{ij}^{\text{edge}} = \mathbf{1}[y_i = y_j \wedge y_i \neq 0].$$

For TrackEmbed/condensation, keep per-hit instance labels. For query/slot models, use Hungarian matching between predicted masks and target clusters.

3.3 Balancing edge cases

The training distribution must include:

- single-hit particles;
- small two- or three-hit particles;
- long tracks with tens/hundreds of hits;
- crossing or overlapping tracks;
- repeated same-pixel hits;
- high-density bursts;
- empty or near-empty windows.

If the loss is naively averaged over hits, long tracks dominate. Use cluster-balanced weighting: each particle contributes approximately the same total weight, independent of hit count.

3.4 Evaluation metrics

For the first pass, do not measure only point-level accuracy. Use instance metrics:

- Adjusted Rand Index (ARI), V-measure, and pairwise F1 for hit grouping;
- split rate: one true particle split into several predictions;
- merge rate: several true particles merged into one prediction;
- energy conservation: summed ToT per predicted particle versus pseudo/true particle;
- latency p50/p95/p99 for windows of different hit counts.

4 Pass 2: variable-size particle classification without labels

After pass 1, each candidate particle is itself a variable-size set:

$$P_j = \{(x_i, y_i, t_i, e_i)\}_{i=1}^{M_j}, \qquad M_j \text{ variable.}$$

The goal is not supervised classification into known labels. The goal is to learn an embedding

$$P_j \mapsto z_j \in \mathbb{R}^D$$

where similar trajectories and energy profiles are close. Then cluster z_j values to discover families.

4.1 Particle encoder: what it physically looks like

A minimal particle encoder is a PointNet/DeepSets encoder: apply the same MLP to every hit, then pool. PointNet and DeepSets are canonical variable-size set encoders [Qi et al., 2017a, Zaheer et al., 2017]. PointNet++ and graph/point-transformer variants add local geometry and usually improve shape sensitivity [Qi et al., 2017b, Wang et al., 2019, Zhao et al., 2021]. Recent detector work with point-set transformers is particularly relevant because it works directly on sparse hit matrices/lists and reports speed and memory advantages over dense methods [Robles et al., 2025].

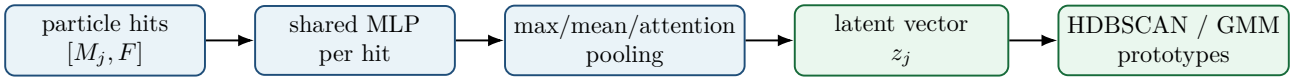

Figure 2: Particle-level encoder. The input particle can contain any number of hits; the latent vector has fixed size and can be clustered.

Recommended hit features for the particle encoder:

$$[\tilde{x}, \tilde{y}, \tilde{t}, \tilde{e}, r, \theta, \Delta t, \text{rank\_by\_time}],$$

where coordinates are centered on the candidate particle, r, θ are optional local polar coordinates, and global attributes such as total ToT, duration, bounding-box size, PCA eigenvalues, and fitted direction are concatenated after pooling.

5 Autoencoders for unsupervised particle grouping

Autoencoders are a good fit here, but only if designed for point sets. A normal image autoencoder would force a dense representation and lose timing/multiplicity.

5.1 Option 1: Point-cloud autoencoder

The simplest unsupervised model is:

$$P_j \xrightarrow{\text{encoder}} z_j \xrightarrow{\text{decoder}} \hat{P}_j.$$

The decoder reconstructs either a fixed number of sampled points or a distribution over points. The reconstruction loss can use Chamfer distance plus energy/time losses:

$$\mathcal{L}_{\text{Chamfer}}(P, \hat{P}) = \sum_{p \in P} \min_{\hat{p} \in \hat{P}} \|p - \hat{p}\|^2 + \sum_{\hat{p} \in \hat{P}} \min_{p \in P} \|\hat{p} - p\|^2.$$

Add

$$\mathcal{L}_{e,t} = \sum |e - \hat{e}| + \lambda_t |t - \hat{t}|.$$

After training, cluster the latent vectors z_j with HDBSCAN, GMM, or k-means.

When to use it. Use this as the first unlabeled particle grouping baseline. It is simple, robust, and does not require class labels.

Caveat. A pure reconstruction loss can learn irrelevant details, such as absolute position or hit count, instead of meaningful trajectory families. Therefore use centered/normalized coordinates and add explicit summary features only after the encoder.

5.2 Option 2: Denoising and masked autoencoder

A denoising autoencoder corrupts the input particle and reconstructs the original:

$$\tilde{P}_j = \text{augment}(P_j), \qquad \hat{P}_j = D(E(\tilde{P}_j)).$$

Useful augmentations are small coordinate jitter, ToT jitter, random hit dropout, time-bin masking, and mild resampling. The learned latent vector becomes more stable than with a vanilla AE.

A masked point autoencoder hides a subset of hits or time bins and asks the model to reconstruct them. This is closely aligned with recent masked point-modeling for particle trajectories, where representation learning is performed directly on sparse 3D point-cloud detector data without labels [Young et al., 2025]. This is one of the most promising self-supervised approaches for your second pass.

5.3 Option 3: DEC/IDEC deep embedded clustering

Deep Embedded Clustering (DEC) pretrains an autoencoder, initializes cluster centers in latent space, then jointly optimizes the encoder and cluster assignments [Xie et al., 2016]. IDEC adds reconstruction preservation to avoid destroying local structure [Guo et al., 2017]. This is useful if you can choose an approximate number of morphology families K.

$$q_{jk} = \frac{(1 + \|z_j - \mu_k\|^2/\alpha)^{-(\alpha+1)/2}}{\sum_l (1 + \|z_j - \mu_l\|^2/\alpha)^{-(\alpha+1)/2}}.$$

DEC sharpens these soft assignments and trains the encoder so that clusters become more compact.

When to use it. Use DEC/IDEC when you want a fixed taxonomy such as 8–20 morphology groups and you can tolerate post-hoc interpretation of cluster IDs.

5.4 Option 4: Variational deep embedding / mixture VAE

A VAE maps a particle to a distribution $q(z|P)$, not just a point. Variational Deep Embedding (VaDE) combines a VAE with a Gaussian mixture prior and directly learns unsupervised clusters [Kingma and Welling, 2014, Jiang et al., 2017]. This is attractive when groups overlap and uncertainty matters.

$$c \sim \text{Categorical}(\pi), \quad z \sim \mathcal{N}(\mu_c, \Sigma_c), \quad P \sim p_\theta(P|z).$$

When to use it. Use VaDE-like models when you expect fuzzy categories: e.g. short-track versus compact-deposit mixtures, or ambiguous overlaps. It gives cluster probabilities rather than hard labels.

5.5 Option 5: contrastive particle embedding

Contrastive learning is often stronger than reconstruction if the desired similarity is known through augmentations. Create two views of the same particle by dropout/jitter/time masking and train them to be close while other particles are far [Chen et al., 2020]. In particle physics, unsupervised and lightly supervised learning is an active direction for representation learning and anomaly detection [Kasieczka et al., 2024].

For your data, positives can be:

- two random hit subsamples of the same particle;
- the same particle with small ToA/ToT jitter;
- the same candidate after two segmentation thresholds;
- synthetic and detector-smeared versions of the same generated particle.

The latent space is then clustered with HDBSCAN or a prototype bank.

Table 3: Autoencoder and self-supervised options for unlabeled particle grouping.

Method	What it learns	Recommendation
Point-cloud AE	latent preserving geometry/time/ToT enough to reconstruct particle	best first unsupervised baseline
Denoising AE	robust latent invariant to missing hits and noise	recommended for real noisy candidates
Masked point AE	representation from reconstructing hidden trajectory pieces	very promising for unlabeled sparse trajectories
DEC/IDEC	encoder + fixed number of latent clusters	good if you want a fixed morphology taxonomy
VaDE / GMM-VAE	probabilistic mixture clusters with uncertainty	good for fuzzy classes and overlaps
Contrastive embedding	similarity based on physics-preserving augmentations	often best for grouping by shape/energy rather than reconstruction detail

6 Concrete second-pass design: ParticleAE-Cluster

Recommended second-pass model

ParticleAE-Cluster = centered point-cloud encoder + masked/denoising reconstruction + contrastive consistency + HDBSCAN or VaDE in latent space. Do not force physical class names during training. Discover groups first, then name them by inspecting prototypes and summary statistics.

6.1 Encoder

For each separated particle, center and normalize coordinates:

$$\bar{x} = x - x_{\text{centroid}}, \quad \bar{y} = y - y_{\text{centroid}}, \quad \bar{t} = t - t_{\min}.$$

Then encode with:

$$\begin{aligned} \text{HitMLP}: &\quad 6 \to 64 \to 64, \\ \text{LocalBlock}: &\quad \text{optional EdgeConv or point-transformer block}, \\ \text{Pooling}: &\quad [\max, \text{mean}, \text{attention}], \\ \text{SummaryConcat}: &\quad \text{duration, total ToT, PCA eigenvalues, fitted direction}, \\ \text{Latent}: &\quad 32\text{–}128 \text{ dimensions}. \end{aligned}$$

6.2 Decoder

Choose one of two decoders:

1. **Point decoder:** latent z generates M_0 canonical points, e.g. $M_0 = 32$ or 64. Train with Chamfer distance against a resampled version of the particle.
2. **Distribution decoder:** latent z predicts a mixture of Gaussian line/curve components in (x, y, t, e). This is more physics-shaped and handles variable hit counts better.

6.3 Clustering and prototype extraction

After training, embed all particles and run:

- HDBSCAN for unknown number of groups and noise/ambiguous candidates;
- GMM/VaDE if you want soft cluster probabilities;
- k-means only as a baseline if K is chosen manually.

For every cluster, store medoids and prototypes: the 10 candidates closest to the cluster center. A domain expert should name these groups after inspecting trajectory shape, total ToT, duration, direction, and size distributions.

7 Experiment design

7.1 E0: deterministic 3D preprocessing audit

Implement only:

$$\texttt{MatrixIndex} \to (x, y), \quad t = \mathrm{ToA} + \delta(\mathrm{FToA}), \quad e = \log(1 + \mathrm{ToT}).$$

No image generation. Build distributions of N hits/window, duplicate pixels, ToT, time span, and local density. This defines the scaling constants for DBSCAN and kNN graphs.

7.2 E1: 3D DBSCAN/HDBSCAN baseline

Run 2D image DBSCAN versus 3D DBSCAN/HDBSCAN on the same windows. Measure split/merge behavior manually on a small validation set. Use parameter stability to select high-confidence pseudo-labels.

7.3 E2: EdgeTrackNet-Tiny reproducing DBSCAN

Train EdgeTrackNet using stable 3D DBSCAN clusters as pseudo-labels. The goal is not physics yet; the goal is to reproduce and smooth the grouping behavior faster and with a learnable metric.

Success criteria. ARI > 0.9 against stable pseudo-labels on held-out windows, fewer obvious split/merge failures than 3D DBSCAN on manually inspected examples, and p95 inference below the target latency.

7.4 E3: synthetic truth and object condensation

Build a simple parametric simulator: generate points from lines, short segments, compact blobs, and diffuse fragments in (x, y, t); assign ToT profiles; add detector noise, duplicate pixels, and time jitter. This produces exact hit-to-particle labels. Train TrackEmbed/CondensationTrackNet with real-style augmentation and test on real data using DBSCAN/manually inspected validation.

7.5 E4: ParticleAE baseline

Take separated particles from E2/E3. Train point-cloud AE and denoising AE. Cluster latent vectors with HDBSCAN. Inspect cluster prototypes. Check whether clusters correspond to meaningful morphology: single hits, compact blobs, straight short tracks, long tracks, high-ToT clusters, fragments, overlaps.

7.6 E5: DEC/VaDE/contrastive grouping

Compare:

- AE + HDBSCAN;
- DEC/IDEC with $K = 8, 12, 16, 24$;
- VaDE with mixture uncertainty;
- contrastive embedding + HDBSCAN.

Evaluate cluster stability under resampling, time-split reproducibility, silhouette/DBCV scores, prototype interpretability, and consistency of energy/ToT distributions.

7.7 E6: live deployment test

Run the complete pipeline on recorded stream replay:

$$\text{parse hits} \rightarrow \text{Pass 1 grouping} \rightarrow \text{Pass 2 embedding/grouping}.$$

Measure p50/p95/p99 latency for empty, median, p95, and burst windows. If burst windows break latency, use a hybrid rule: local pool dense regions, cap token count, or fall back to 3D DBSCAN on a restricted region.

8 SOTA map

Table 4: Relevant SOTA directions and how they map to the two tasks.

Area	Representative work	Relevance
Timepix/TPX3 processing	Timepix3 chip, DPE, TraX, direction reconstruction [Poikela et al., 2014, Marek et al., 2023, Oancea et al., 2024, Mánek et al., 2021]	Confirms that sparse ToA/ToT tracking and radiation-field products are the native domain.
Density clustering	DBSCAN, HDBSCAN, ST-DBSCAN [Ester et al., 1996, McInnes et al., 2017, Birant and Kut, 2007]	Baseline and pseudo-label source for pass 1.
Point-set learning	PointNet, PointNet++, Deep Sets, Set Transformer [Qi et al., 2017a,b, Zaheer et al., 2017, Lee et al., 2019]	Variable-size hit/particle encoders; no dense images required.
Local point geometry	DGCNN/EdgeConv and Point Transformer [Wang et al., 2019, Zhao et al., 2021]	Learns local track geometry, useful for both grouping and particle embeddings.
Detector point-set SOTA	Point-set transformer for sparse detector hit clustering [Robles et al., 2025]	Strong evidence that sparse point-set models are appropriate for detector hit data.
Learned instance grouping	Object condensation and HGCAL applications [Kieseler, 2020, Qasim et al., 2021, Matousek et al., 2025]	Most relevant end-to-end method for variable number of particles.
Query/slot reconstruction	Slot Attention, TrackFormers [Locatello et al., 2020, Caron et al., 2024]	Candidate second-generation model for variable particles via learned slots.
Autoencoder clustering	DEC, IDEC, VaDE, deep clustering surveys [Xie et al., 2016, Guo et al., 2017, Jiang et al., 2017, Min et al., 2018, Lu et al., 2024]	Core toolkit for unlabeled particle-family discovery.
Self-supervised trajectories	Masked point modeling / PoLAr-MAE [Young et al., 2025]	Most aligned unlabeled pretraining strategy for sparse 3D trajectories.
Unsupervised particle physics	Recent unsupervised/lightly supervised learning review [Kasieczka et al., 2024]	Context for anomaly/grouping use when labels are absent.

9 Recommended concrete implementation plan

9.1 First four weeks

1. Implement a deterministic parser and 3D DBSCAN/HDBSCAN baseline in (x, y, t, e).
2. Generate stable pseudo-labels by varying DBSCAN parameters.
3. Train EdgeTrackNet-Tiny on pseudo-labels.
4. Build visualization: raw hits colored by DBSCAN, colored by EdgeTrackNet, and disagreements.

9.2 Second four weeks

1. Build a simple simulator for lines, blobs, short tracks, overlaps, single-hit particles, and noise.

2. Train TrackEmbed/CondensationTrackNet on synthetic truth and fine-tune on real pseudo-labels.
3. Extract particle candidates and compute trajectory summaries.
4. Train ParticleAE and contrastive ParticleEmbed.
5. Run HDBSCAN/VaDE and produce prototype galleries for expert naming.

9.3 Minimal model specifications

Table 5: Practical model sizes for a laptop-friendly first prototype.

Model	Parameter scale	Notes
EdgeTrackNet-Tiny	0.1–0.5M	fastest pass-1 model; edge links + connected components
TrackEmbed-Tiny	0.3–1.0M	learned embedding; more flexible than edge-only
CondensationTrackNet-Small	0.5–2.0M	best if synthetic truth exists
ParticleAE-Tiny	0.2–1.0M	pass-2 unsupervised latent grouping
ParticleVaDE-Small	0.5–2.0M	soft group probabilities and uncertainty

10 Final recommendation

The most robust path is not a single monolithic transformer or image CNN. It is a two-pass sparse point-cloud system:

1. **Pass 1: EdgeTrackNet-Tiny first.** It is the most direct neural analogue of 3D DBSCAN: train on stable DBSCAN pseudo-labels, predict local same-particle links, and recover variable-size particles by connected components. Then move to TrackEmbed or object condensation once synthetic labels are available.
2. **Pass 2: ParticleAE/ParticleEmbed.** Encode each separated particle as a fixed latent vector using a variable-size point-cloud encoder. Train with denoising/masked reconstruction and contrastive augmentations. Cluster latent vectors with HDBSCAN first, then try DEC/IDEC or VaDE if a fixed or soft taxonomy is desired.
3. **Use autoencoders carefully.** They are useful for representation learning, but reconstruction alone may focus on irrelevant details. Center coordinates, normalize time, use ToT-aware losses, and validate clusters by stability and prototype interpretability.
4. **Keep DBSCAN in the loop.** It should remain as a teacher, diagnostic, and fallback. The neural model should learn a better metric and should be validated against both DBSCAN stability and synthetic truth.

References

T. Poikela, J. Plosila, T. Westerlund, M. Campbell, M. De Gaspari, X. Llopart, V. Gromov, R. Kluit, M. van Beuzekom, F. Zappon, et al. Timepix3: a 65k channel hybrid pixel readout chip with simultaneous ToA/ToT and sparse readout. *Journal of Instrumentation*, 9(05):C05013, 2014. doi: 10.1088/1748-0221/9/05/C05013.

Petr Mánek, Benedikt Bergmann, Petr Burian, Declan Garvey, Lukáš Meduna, Stanislav Pospíšil, Petr Smolyanskiy, and Eoghan White. Improved algorithms for determination of particle directions with Timepix3. *arXiv preprint arXiv:2111.00624*, 2021.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.

Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software*, 2(11):205, 2017. doi: 10.21105/joss.00205.

Derya Birant and Alp Kut. ST-DBSCAN: An algorithm for clustering spatial-temporal data. *Data & Knowledge Engineering*, 60(1):208–221, 2007. doi: 10.1016/j.datak.2006.01.013.

Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph CNN for learning on point clouds. In *ACM Transactions on Graphics*, volume 38, page 146, 2019. doi: 10.1145/3326362.

Jan Kieseler. Object condensation: one-stage grid-free multi-object reconstruction in physics detectors, graph and image data. *European Physical Journal C*, 80:886, 2020. doi: 10.1140/epjc/s10052-020-08461-2.

Shah Rukh Qasim, Kenneth Long, Jan Kieseler, Maurizio Pierini, and Raheel Nawaz. Multi-particle reconstruction in the high granularity calorimeter using object condensation and graph neural networks. In *EPJ Web of Conferences*, volume 251, page 03072, 2021. doi: 10.1051/epjconf/202125103072.

Michael Matousek et al. AI-assisted object condensation clustering for calorimeter shower reconstruction at CLAS12. *arXiv preprint arXiv:2503.11277*, 2025.

Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems*, volume 33, pages 11525–11538, 2020.

Sascha Caron et al. TrackFormers: In search of transformer-based particle tracking for the high-luminosity LHC era. *arXiv preprint arXiv:2407.07179*, 2024.

Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 652–660, 2017a.

Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, volume 30, 2017b.

Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip H. S. Torr, and Vladlen Koltun. Point transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 16259–16268, 2021.

Edgar E. Robles, Alejandro Yankelevich, Wenjie Wu, Jianming Bian, and Pierre Baldi. Particle hit clustering and identification using point set transformers in liquid argon time projection chambers. *Journal of Instrumentation*, 20(07):P07030, 2025.

Sam Young, Zhen Liu, et al. Particle trajectory representation learning with masked point modeling. *arXiv preprint arXiv:2502.02558*, 2025.

Junyuan Xie, Ross Girshick, and Ali Farhadi. Unsupervised deep embedding for clustering analysis. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 478–487, 2016.

Xifeng Guo, Long Gao, Xinwang Liu, and Jianping Yin. Improved deep embedded clustering with local structure preservation. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 1753–1759, 2017.

Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.

Zhuxi Jiang, Yin Zheng, Huachun Tan, Bangsheng Tang, and Hanning Zhou. Variational deep embedding: An unsupervised and generative approach to clustering. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 1965–1972, 2017.

Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, pages 1597–1607, 2020.

Gregor Kasieczka et al. Unsupervised and lightly supervised learning in particle physics. *arXiv preprint arXiv:2403.13676*, 2024.

Lukáš Marek, Carlos Granja, Jan Jakubek, Jan Ingerle, Daniel Turecek, Marco Vuolo, and Cristina Oancea. Data Processing Engine (DPE): Data analysis tool for particle tracking and mixed radiation field characterization with pixel detectors Timepix. *arXiv preprint arXiv:2310.15723*, 2023.

C. Oancea, L. Marek, M. Vuolo, J. Jakubek, E. Soharová, J. Ingerle, D. Turecek, M. Andrlik, V. Vondracek, T. Baca, et al. TraX engine: Advanced processing of radiation data acquired by Timepix detectors in space, medical, educational and imaging applications. *arXiv preprint arXiv:2410.10242*, 2024.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 3744–3753, 2019.

Erxue Min, Xifeng Guo, Qiang Liu, Gen Zhang, Jianjing Cui, and Jun Long. A survey of clustering with deep learning: From the perspective of network architecture. *IEEE Access*, 6:39501–39514, 2018. doi: 10.1109/ACCESS.2018.2855437.

Ying Lu et al. A survey on deep clustering: From the prior perspective. *arXiv preprint arXiv:2406.19602*, 2024.